Lab 3 Report

Machine Learning for Hardware Design

Authors: Zach Dixon, Jongwoo Kim

3.3.1 State Machine MAC

(a) Module functionality and accumulator control

staged_mac implements the multiply-accumulate core used to compute a single output value of the conv1 layer (a 5×5×3×32 Conv2D layer, selected from the Lab 2 C++ reference model). For one output pixel, the module receives a stream of (activation, weight) pairs over the AXI-Stream slave interface. Each accepted beat multiplies an 8-bit signed activation by an 8-bit signed weight and adds the signed 32-bit result into a running accumulator register. SD_AXIS_TLAST marks the final beat of the group, at which point the accumulated result is presented on the AXI-Stream master interface.

The accumulator does need to be loaded with a non-zero initial value, and this is handled through SD_AXIS_TUSER. Examining the Lab 2 reference implementation (Convolutional.cpp, computeNaive()) shows that each output channel carries its own bias term, and the accumulation for a given output pixel begins as sum = bias[oc] before any multiply-accumulate terms are added. To reproduce this behavior, the first beat of a group can be marked with SD_AXIS_TUSER = '1', in which case staged_mac does not multiply that beat at all; instead it sign-extends the full 16-bit SD_AXIS_TDATA value directly into the 32-bit accumulator as the initial value. All subsequent beats in the group are then treated as ordinary multiply-accumulate operations added on top of that bias.

The accumulator's lifecycle therefore has three distinct control states, all gated by the FSM: it is **loaded** with the bias value when SD_AXIS_TUSER = '1', it **accumulates** the running product on every other accepted beat, and — critically — it is explicitly **cleared back to zero** once the group's result has been consumed (MO_AXIS_TREADY = '1' in the SEND_RESULT state), before the FSM returns to WAIT_FOR_VALUES. This clear step is not optional: without it, the accumulator would retain the previous group's final value, and the next independent group (which typically does not begin with SD_AXIS_TUSER = '1') would silently add its own terms on top of that stale value. The accumulator only ever updates while state = WAIT_FOR_VALUES and SD_AXIS_TVALID = '1' simultaneously; SD_AXIS_TREADY is asserted only in that same state, so no beat is ever accepted — and therefore no update ever occurs — while the previous group's result is still being drained.

(b) Sample inputs and outputs

A small worked example, using a 2×2×1 filter with no bias, illustrates the basic operation:

input = [[1, 2], [3, 4]]
weight = [[1, 0], [0, 1]]
expected output = 1*1 + 2*0 + 3*0 + 4*1 = 5

AXI-Stream beat sequence (TDATA = 8-bit activation concatenated with 8-bit weight):

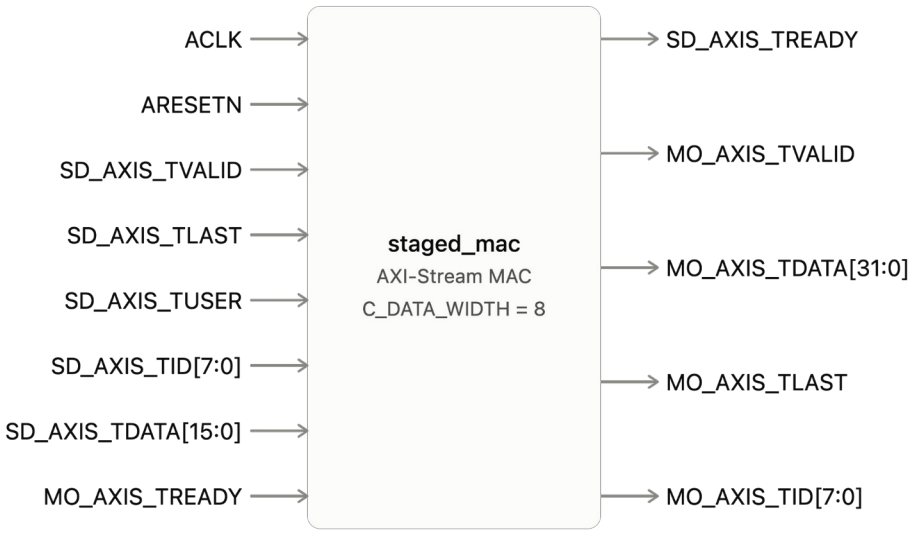
1) TDATA = 0x0101 (1*1), TUSER=0, TLAST=0
2) TDATA = 0x0200 (2*0), TUSER=0, TLAST=0
3) TDATA = 0x0300 (3*0), TUSER=0, TLAST=0
4) TDATA = 0x0401 (4*1), TUSER=0, TLAST=1 -> MO_AXIS_TDATA = 0x00000005

At the actual conv1 scale (5×5×3 kernel, with bias), a group consists of one bias-load beat followed by N = 75 multiply-accumulate beats. For example, with bias = 5 and every product equal to 2×3 = 6:

Beat 1: TUSER=1, TDATA=0x0005 -> loads bias, acc = 5
Beats 2..76: TDATA=0x0203 (2*3=6) each -> acc accumulates by +6 per beat
Beat 76: TLAST=1 -> MO_AXIS_TDATA = 5 + 75*6 = 455 = 0x000001C7

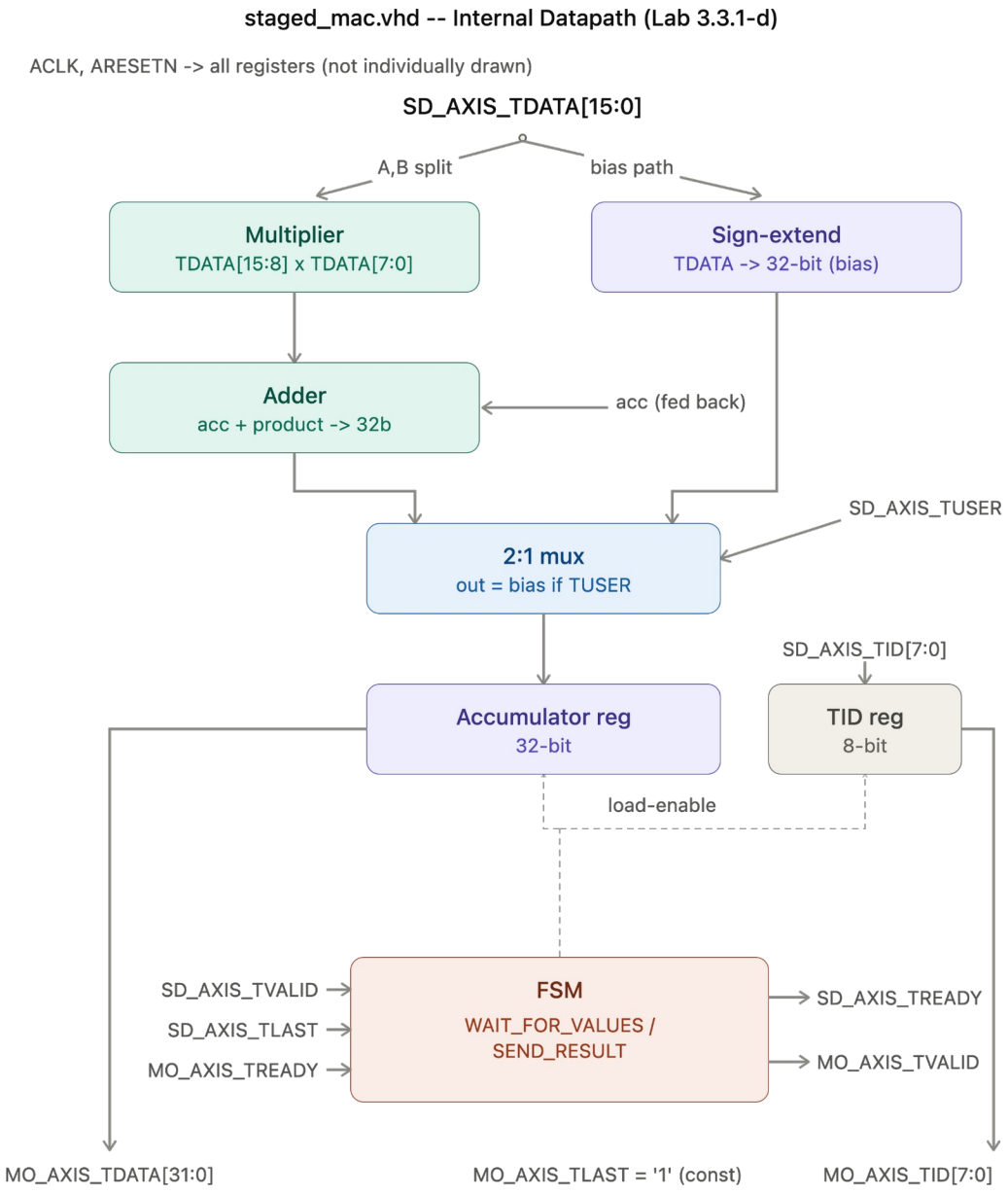
(c) Module interface

The external interface of staged_mac — all AXI-Stream slave and master ports, ACLK/ARESETN, and their bit widths — is shown as a symbol/black-box diagram.



(d) Schematic prior to HDL implementation

The internal datapath — the 8-bit×8-bit multiplier, the sign-extension path used for the bias load, the 32-bit adder with its accumulator feedback, the TUSER-selected 2:1 mux, the accumulator and TID registers, and the two-state FSM that governs when those registers update and when results are presented — is shown below, with every internal signal name and bit width labeled.



(e) Bit-width calculation and overflow argument

Signal	Width	Justification
Activation operand	8 bits	int8 quantization, as specified for this lab
Weight operand	8 bits	int8 quantization, as specified for this lab

Packed input (SD_AXIS_TDATA)	16 bits	Two 8-bit operands concatenated onto one AXI-Stream beat
Product (before resize)	16 bits	Signed 8-bit × 8-bit multiplication produces a result of width 8+8=16 bits
Accumulator / MO_AXIS_TDATA	32 bits	Chosen generously; justified not to overflow below
SD_AXIS_TID / MO_AXIS_TID	8 bits	Matches the AXI-Stream ID convention used to tag each group, passed through unmodified

Each operand is an 8-bit signed integer (int8, range −128 to 127), so the largest-magnitude product occurs at −128 × −128 = 16384 = 2^14. For conv1, each output pixel accumulates N = 5×5×3 = 75 such products on top of a bias value. A conservative upper bound on the magnitude of the accumulator is therefore:

$$|ACC|max <= |bias| + N * 2^14$$
$$<= 32768 + 75 * 16384$$
$$= 1,261,568$$

This is only about 0.06% of the 32-bit signed range (±2,147,483,647), so staged_mac's 32-bit accumulator never approaches overflow for conv1. Generalizing across every conv layer in the reference model — the largest being conv2 with N = 800 — the bound still only reaches roughly 0.6% of the available range: 800 * 16384 + 32768 = 13,139,968

Solving for the group size at which overflow could first occur, N > 2^31 / 2^14 ≈ 131,072, shows that no layer in this model comes remotely close to requiring saturation or overflow handling, confirming that a plain 32-bit signed accumulator with no saturation logic is sufficient.

(f) Testing strategy and justification

staged_mac was verified with a self-checking AXI-Stream Tcl testbench (staged_mac_tb.tcl) built around a check_result procedure that reads MO_AXIS_TDATA at the correct cycle and prints a PASS/FAIL line against a hand-computed expected value, rather than relying on manual waveform inspection. Twelve test cases were run, all passing:

1. Single-beat group (basic multiply-accumulate)
2. Two-beat group (accumulation across multiple beats)
3. First of two independent single-beat groups issued back-to-back with zero idle cycles
4. Second of two independent single-beat groups issued back-to-back with zero idle cycles
5. Initial bias load (TUSER) followed by a multiply-accumulate
6. Multi-beat group with idle gaps between beats (SD_AXIS_TVALID deasserted mid-group)
7. Back-pressure: MO_AXIS_TREADY held low while a result is pending
8. conv1-scale accumulation: bias load followed by N = 75 continuous beats
9. Maximum-magnitude positive product (127 × 127)
10. Maximum-magnitude negative-operand product (−128 × −128)
11. Minimal group: a single beat with TLAST asserted in the same cycle as the data, no separate bias-load beat
12. Negative accumulator result (127 × −128), verifying correct 32-bit sign extension

This set is considered sufficient because it independently exercises every distinct code path in the design: the bias-load path, the accumulate path at both minimal and full layer-scale granularity, both AXI-Stream backpressure and idle-cycle handling, both signed extremes of the int8 operand range, and — critically — the accumulator-reset behavior between independent groups, a case that surfaced and caught a genuine accumulator-carryover bug during development (an earlier revision failed to clear the accumulator when returning to WAIT_FOR_VALUES, causing test cases 3–4 to fail until it was fixed).

(g) Synthesis, critical path, and throughput

Each individual multiply-accumulate operation takes exactly 1 clock cycle: staged_mac accepts one (activation, weight) beat and updates the accumulator every cycle while in WAIT_FOR_VALUES. Using the conv1 layer (5×5×3×32 kernel, 60×60×32 output), each output pixel requires N = 5×5×3 = 75 such

multiply-accumulate operations, plus one additional cycle to present the result in SEND_RESULT, so each output pixel costs N + 1 = 76 cycles.

Timing. Post-route static timing analysis reports a Worst Negative Slack (WNS) of 0.834 ns against the 200 MHz constraint, so the design meets timing with margin: f_max = 1000 / (5.000 − 0.834) ≈ **240.0 MHz**.

```
| Design Timing Summary
| ---------------------

    WNS(ns)      TNS(ns)  TNS Failing Endpoints  TNS Total Endpoints      WHS(ns)      THS(ns)  THS Failing Endpoints  THS Total End
    -------      -------  ---------------------  -------------------      -------      -------  ---------------------  -------------
      0.834        0.000                      0                   73        0.291        0.000                      0
```

Critical path. The longest path runs from state_reg/C to acc_reg[5]/D — the FSM's state register feeding the accumulator's load-enable logic. The path crosses only one logic level (one LUT4), with 84% of the 3.778 ns data path delay coming from routing rather than logic, since the design's low utilization gave the placer little incentive to place these registers close together.

```
Max Delay Paths
--------------------------------------------------------------------------------------
Slack (MET) :             0.834ns  (required time - arrival time)
  Source:                 state_reg/C
                            (rising edge-triggered cell FDRE clocked by main  {rise@0.000ns fall@0.050ns period=5.000ns})
  Destination:            acc_reg[5]/D
                            (rising edge-triggered cell FDRE clocked by main  {rise@0.000ns fall@0.050ns period=5.000ns})
  Path Group:             main
  Path Type:              Setup (Max at Slow Process Corner)
  Requirement:            5.000ns  (main rise@5.000ns - main rise@0.000ns)
  Data Path Delay:        3.778ns  (logic 0.610ns (16.145%)  route 3.168ns (83.855%))
  Logic Levels:           1  (LUT4=1)
  Clock Path Skew:        -0.068ns (DCD - SCD + CPR)
    Destination Clock Delay (DCD):    4.599ns = ( 9.599 - 5.000 )
    Source Clock Delay      (SCD):    5.124ns
    Clock Pessimism Removal (CPR):    0.457ns
  Clock Uncertainty:      0.035ns  ((TSJ^2 + TIJ^2)^1/2 + DJ) / 2 + PE
    Total System Jitter     (TSJ):    0.071ns
    Total Input Jitter      (TIJ):    0.000ns
    Discrete Jitter          (DJ):    0.000ns
    Phase Error              (PE):    0.000ns

    Location             Delay type                Incr(ns)  Path(ns)    Netlist Resource(s)
  -------------------------------------------------------------------    -------------------
                         (clock main rise edge)       0.000     0.000 r
    K19                                               0.000     0.000 r  ACLK (IN)
                         net (fo=0)                   0.000     0.000    ACLK
    K19                  IBUF (Prop_ibuf_I_O)         0.938     0.938 r  ACLK_IBUF_inst/O
                         net (fo=1, routed)           2.205     3.143    ACLK_IBUF
    BUFGCTRL_X0Y16       BUFG (Prop_bufg_I_O)         0.101     3.244 r  ACLK_IBUF_BUFG_inst/O
                         net (fo=41, routed)          1.880     5.124    ACLK_IBUF_BUFG
    SLICE_X113Y40        FDRE                                        r  state_reg/C
```

Resource utilization.

Resource	Used	Available	Utilization
Slice LUTs	118	53,200	0.22%

Slice Registers	41	106,400	0.04%
DSP48	0	220	0.00%

No DSP48 blocks are used, since the 8-bit × 8-bit signed multiplier is small enough for Vivado to implement efficiently in LUT logic.

```
1. Slice Logic
--------------

+-------------------------+------+-------+-----------+-------+
|        Site Type        | Used | Fixed | Available | Util% |
+-------------------------+------+-------+-----------+-------+
| Slice LUTs              |  118 |     0 |     53200 |  0.22 |
|   LUT as Logic          |  118 |     0 |     53200 |  0.22 |
|   LUT as Memory         |    0 |     0 |     17400 |  0.00 |
| Slice Registers         |   41 |     0 |    106400 |  0.04 |
|   Register as Flip Flop |   41 |     0 |    106400 |  0.04 |
|   Register as Latch     |    0 |     0 |    106400 |  0.00 |
| F7 Muxes                |    0 |     0 |     26600 |  0.00 |
| F8 Muxes                |    0 |     0 |     13300 |  0.00 |
+-------------------------+------+-------+-----------+-------+
```

```
4. DSP
------

+-----------+------+-------+-----------+-------+
| Site Type | Used | Fixed | Available | Util% |
+-----------+------+-------+-----------+-------+
| DSPs      |    0 |     0 |       220 |  0.00 |
+-----------+------+-------+-----------+-------+
```

Timing caveat. Although setup timing is met, the post-route pulse-width check fails on 41 of 42 endpoints (WPWS = −0.450 ns). This is not a design flaw: zedboard.xdc defines the clock as -waveform {0.000 0.050}, a 1% duty cycle far

narrower than the standard 50% ({0.000 2.500}), which is a constraint-file issue independent of the setup-timing results above.

THS(ns)	THS Failing Endpoints	THS Total Endpoints	WPWS(ns)	TPWS(ns)	TPWS Failing Endpoints	TPWS Total Endpoints
0.000	0	73	-0.450	-18.450	41	42

Throughput.

- Time per output pixel: 76 × (1000/240.0) ns ≈ 316.6 ns
- Total MAC operations for conv1: 60 × 60 × 32 × 75 = **8,640,000**
- Total time for the full layer: 115,200 outputs × 76 cycles ÷ 240.0 MHz ≈ **36.5 ms**
- Effective throughput: 8,640,000 ÷ 0.0365 s ≈ **236.9 MMAC/s**

Scaling to multiple PEs. Under the stated assumption of negligible memory/data-movement overhead, P parallel PEs would scale throughput linearly to approximately P × 236.9 MMAC/s.

Is this estimate reasonable? Only as an optimistic upper bound. The single-PE number is credible, but linear scaling breaks down as P grows: every additional PE needs its own data stream every cycle, and real memory bandwidth cannot supply an unbounded number of PEs at full rate. Throughput would plateau once memory bandwidth, not compute, becomes the bottleneck.

3.4 PIPELINING

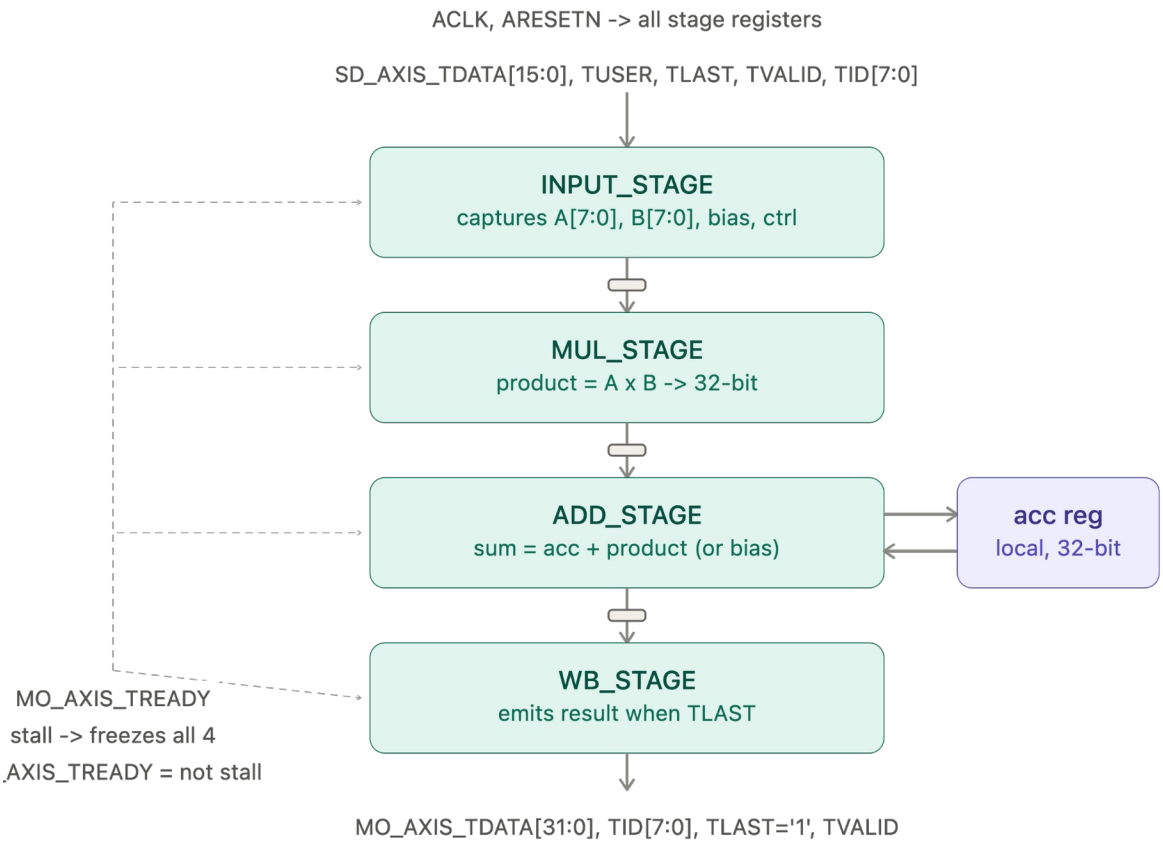
(a) Pipeline register placement and approach

Starting from staged_mac's single-cycle datapath — where operand splitting, the multiply, the accumulate, and the mux all happen combinationally within one clock before landing in the accumulator register — the natural break points for pipelining are the boundaries between those already-distinct operations. This gives four stages:

1. **INPUT_STAGE** — captures and splits SD_AXIS_TDATA into the two 8-bit operands, sign-extends the full 16-bit value for the bias path, and latches SD_AXIS_TUSER/SD_AXIS_TLAST/SD_AXIS_TID.

2. **MUL_STAGE** — computes the signed 8×8 product, resizes it to 32 bits, and passes the bias value and control signals through unchanged.
3. **ADD_STAGE** — computes acc + product (or selects the bias value directly when TUSER='1'), and updates a single local accumulator register.
4. **WB_STAGE** — presents the final result on MO_AXIS_TDATA/MO_AXIS_TID/MO_AXIS_TLAST/MO_AXIS_TVALID once the group's last beat has propagated through.

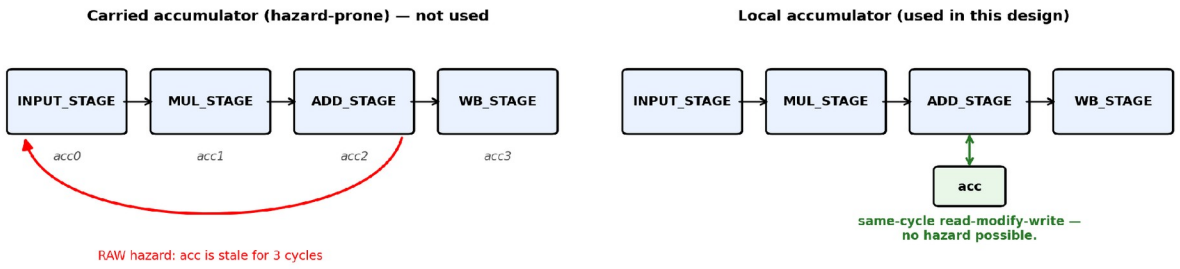
Pipeline registers are inserted at the three stage boundaries (INPUT_STAGE ↔ MUL_STAGE, MUL_STAGE ↔ ADD_STAGE, ADD_STAGE ↔ WB_STAGE), giving a total pipeline depth of four cycles from beat capture to result output. The overall structure, including where each register boundary sits, is shown below.



The key design decision was keeping the running accumulator **local to ADD_STAGE**, updated by direct read-modify-write each cycle, rather than carrying its value through the pipeline registers as part of each beat's payload. Carrying the accumulator as pipelined data — as in an initial teammate design that

used this approach — creates a RAW hazard whenever two beats of the same group are issued closer together than the pipeline depth, which is exactly the common case for a continuous input stream. A comparison of the hazard-prone carried-accumulator design against the hazard-free local-accumulator design used here is shown below.

Figure 2: Carried vs. local accumulator



Keeping acc local to ADD_STAGE removes the RAW hazard by construction.

Maximum achievable depth. ADD_STAGE's accumulate step has to stay one cycle, since acc is read and written in the same cycle and splitting it would bring back the hazard we're avoiding. MUL_STAGE is the only stage with room to grow: the DSP48 already used for the multiply has its own internal input/multiply/output registers, so exposing those as extra stages would push the design to roughly 5-6 stages. That's not worth doing here though — post-route timing (3.4c) already shows the critical path sitting inside the DSP48 slice itself, not at a stage boundary, so splitting MUL_STAGE further would add latency without raising f_max.

(b) Implementation and testing

piped_mac.vhd implements the 4-stage pipeline (INPUT_STAGE/MUL_STAGE/ADD_STAGE/WB_STAGE) in VHDL, reusing the multiply-accumulate logic already verified in staged_mac.

It was tested with the same self-checking Tcl testbench approach, adjusted for the pipeline's 3-cycle latency. 13 test cases passed, including all staged_mac edge cases (int8 extremes, back-pressure, bias load) plus two pipeline-specific cases: two independent groups issued with zero gap between them, and a full N=75-beat group streamed with zero gaps.

These two cases are exactly where a carried-accumulator design would produce a RAW hazard. Both passing confirms the local-accumulator design is hazard-free, which is why this test set is sufficient.

(c) Synthesis, maximum frequency, and throughput

Post-route timing analysis of piped_mac gives a Worst Negative Slack (WNS) of 0.511 ns against the 200 MHz constraint (note: this required adding a use_dsp attribute to force the multiplier into a DSP48 slice, since without it the LUT-based multiplier failed timing). This gives a maximum frequency of:

f_max = 1000 / (5.000 − 0.511) ≈ **222.8 MHz**

Using the same conv1 layer as before (5×5×3×32 kernel, 60×60×32 output, N=75 MACs per output pixel), piped_mac accepts one beat per cycle with no per-group bubble (verified by the zero-gap test cases in 3.4(b)), so a group of N beats takes exactly N = 75 cycles, and the whole layer takes approximately 115,200 × 75 = 8,640,000 cycles:

- Total time for conv1: 8,640,000 cycles ÷ 222.8 MHz ≈ **38.78 ms**
- Effective throughput: 8,640,000 MACs ÷ 0.03878 s ≈ **222.8 MMAC/s**

Interestingly, this is slightly *lower* than staged_mac's 236.9 MMAC/s. Pipelining removed the 1-cycle bubble between groups, but for N=75 that bubble only cost staged_mac about 1.3% of its throughput, while the extra stage registers and stall-control logic in piped_mac lowered its maximum clock frequency by about 7.4% (240.0 MHz → 222.8 MHz). For a layer this large, the clock-frequency cost of pipelining outweighs the bubble it removes.

(d) Pipeline hazards

With a 4-stage pipeline
(INPUT_STAGE → MUL_STAGE → ADD_STAGE → WB_STAGE),

a naive design that carries the running accumulator value through the pipeline registers would have a RAW (read-after-write) hazard: a beat's updated accumulator value is not available again until it has propagated all the way to ADD_STAGE, so any later beat that reads the accumulator sooner than that would read a stale value.

This hazard would require attention whenever two beats of the *same* accumulation group are issued fewer cycles apart than the pipeline depth. For conv1, beats within

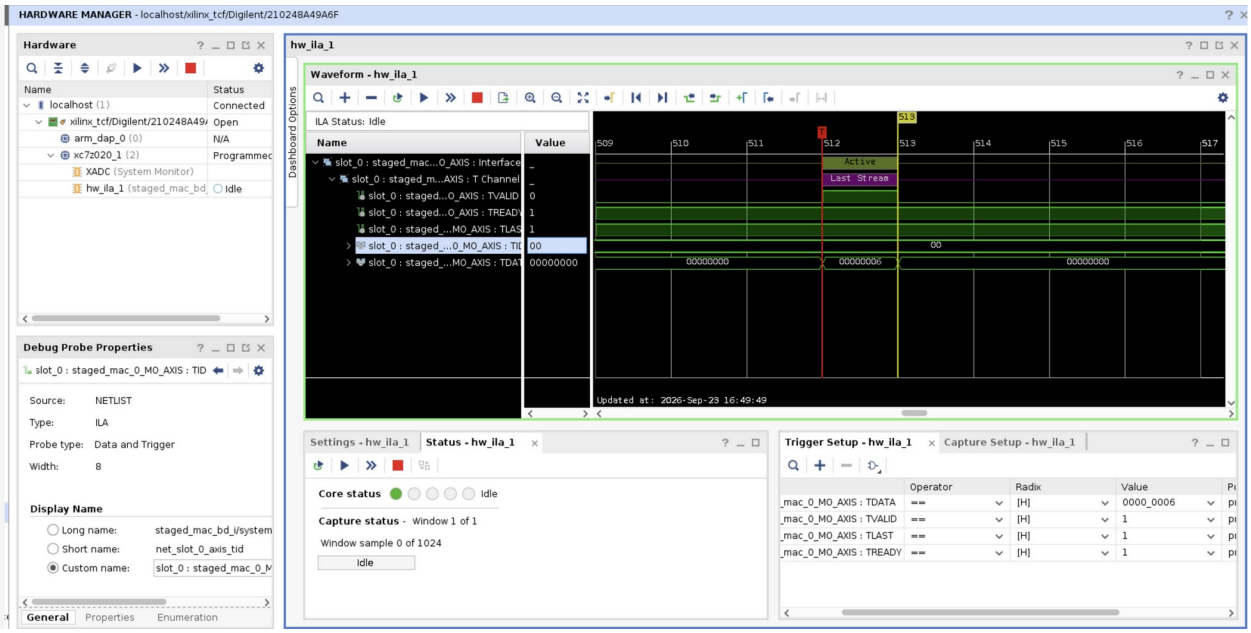
a group are streamed one per cycle (gap = 1 cycle), which is far less than the 4-cycle depth, so the hazard would occur on essentially every beat after the first — 74 out of 75 beats per group (≈98.7%).

Because this hazard would occur on nearly every beat rather than as a rare edge case, piped_mac avoids it by design: the accumulator is kept local to ADD_STAGE and updated by direct read-modify-write each cycle, rather than being carried through the pipeline as data. This was verified in 3.4(b) with back-to-back and continuous-streaming test cases, both of which passed.

Hazards in the actual design. The only state shared between beats is acc itself, and it's read-modify-written in a single cycle inside ADD_STAGE, so this hazard can't actually happen in our design — not just at 4 stages, but even at the maximum depth from 3.4(a). Splitting MUL_STAGE further only changes how long a beat takes to reach ADD_STAGE; it doesn't change that ADD_STAGE itself stays one cycle with a local accumulator, so the hazard-free argument holds regardless of pipeline depth.

5.6 ILA Verification of Team Number

A System ILA core was added to the staged_mac block design, probing MO_AXIS. The trigger was set to MO_AXIS_TDATA == 6 with TVALID, TLAST, and TREADY all high, matching the team-number output beat (ILA_TeamNumber in the C++ test program, expected/output = 6). The capture below shows the trigger firing exactly on that beat, confirming the correct value is present on the bus with all three handshake signals active at the same cycle.



This ILA was added independently to our own build rather than pulled from a teammate's branch; since both designs share the same staged_mac interface, the capture is equivalent either way.

5.7 Testing Strategy and Status

The C++ test suite (software_testing/src/main.cpp) checks staged_mac output against a software-computed reference: a single-beat group, a multi-beat group, two groups in a row (the accumulator must reset between them), both int8 sign extremes, a negative result, and a full N=75 convolution group. On the ZedBoard, all 8 hardware tests passed. The AXI FIFO in this design cannot drive TUSER and always accepts the result, so the board test cannot load a bias or apply back-pressure; those two paths are covered only by the simulation in 3.3.1(f). Together, the 8/8 board tests and the 12/12 (staged_mac) and 13/13 (piped_mac) simulation results cover every distinct behavior of the design: bias load, back-to-back groups, back-pressure, and both int8 sign extremes.

Performance was measured with XTime_GetTime around the N=75 group: 16.557 µs, or ≈ 4.53 MMAC/s. This is well below the synthesis f_max estimate (≈ 236.9 MMAC/s) because the measurement includes MMIO polling overhead from sending each operand as a separate AXI-Stream transaction, not a back-to-back streaming transfer.

Design status summary:

Design	Simulation	Synthesis (Fmax)	HW Test
staged_mac	12/12 passed	240.0 MHz	8/8 passed, ILA captured
piped_mac	13/13 passed	222.8 MHz	Not board-integrated (sim/synth only, per lab scope)